Plan de Acción Integral para el Desarrollo de un Proyecto de Portafolio en Ingeniería de Datos

La consolidación de un portafolio de ingeniería de datos contemporáneo requiere una demostración exhaustiva de habilidades técnicas y arquitectónicas que trascienden la simple ejecución de tutoriales básicos. Los gerentes de contratación y los líderes técnicos buscan evidencia empírica de que un candidato comprende la complejidad del mundo real, incluyendo el manejo de casos límite, la tolerancia a fallos, la optimización de costos, el monitoreo y las prácticas de despliegue continuo[1]. Este documento presenta un análisis profundo y un plan de acción exhaustivo y estructurado para construir un proyecto de portafolio de nivel avanzado. La arquitectura propuesta integra un repositorio altamente estructurado, un ecosistema de *Data Lakehouse* local basado en contenedores, mecanismos rigurosos de control de calidad de datos (*Data Quality*), evaluación de latencia de red para futuras migraciones a la nube, y un enfoque vanguardista de control de versiones aplicado a la inteligencia de negocios (BI) mediante formatos de texto plano.

A través de la adopción de herramientas modernas y estándares empresariales, este informe detalla minuciosamente los pasos arquitectónicos, los archivos requeridos y las áreas de investigación necesarias para articular un proyecto que comunique de manera inequívoca competencia técnica, madurez en ingeniería de software y una visión estratégica de la gestión de datos.

Fase 1: Gobernanza, Estructuración y Buenas Prácticas del Repositorio

El repositorio de código actúa como la carta de presentación principal de cualquier ingeniero de datos. Los equipos de evaluación técnica determinan la madurez de un candidato observando su capacidad para organizar, documentar, proteger y estandarizar el código antes de examinar la lógica algorítmica de sus canalizaciones[2]. Un repositorio desordenado, con credenciales expuestas o carente de instrucciones de ejecución, sugiere una falta de rigor que, en entornos de producción, se traduce inevitablemente en una costosa deuda técnica.

Arquitectura de Directorios y Separación de Responsabilidades

La estructura del repositorio debe ser predecible, segura de revisar y sencilla de ejecutar en un entorno local con conjuntos de datos reducidos o sintéticos[2]. La separación estricta de responsabilidades es un pilar fundamental en el diseño de software; por lo tanto, el código fuente, las consultas de transformación, las pruebas automatizadas, la documentación técnica y los datos de muestra deben residir en directorios lógicamente aislados[2]. Esta organización modular no solo facilita la navegación por parte de revisores externos, sino que también permite el escalado del proyecto a medida que se integran nuevos componentes.

A continuación, se detalla la estructura de directorios recomendada, junto con su justificación

técnica y operativa:

Directorio	Propósito Arquitectónico	Contexto, Origen y Justificación Técnica
src/	Lógica de canalizaciones y módulos de Python.	Contiene el código fuente principal y los scripts reutilizables. La extracción de la lógica de procesamiento desde entornos interactivos (como *Jupyter Notebooks*) hacia módulos de Python estructurados demuestra madurez en la ingeniería de software, permitiendo que el código sea testeable y reutilizable[2].
sql/	Transformaciones, modelos analíticos y consultas.	Aísla la lógica declarativa de bases de datos del código imperativo de orquestación. Esta separación facilita la auditoría de transformaciones y sienta las bases para la integración de herramientas de ingeniería analítica como *dbt* (data build tool)[2].
tests/	Pruebas unitarias, de integración y comprobaciones.	Alberga los scripts de *Pytest* y las configuraciones de validación de estado de *Great Expectations*. Centralizar las pruebas garantiza la integridad continua del código frente a refactorizaciones, previniendo regresiones silenciosas en la lógica de

		transformación[2].
data/	Conjuntos de datos locales y muestras representativas.	Se subdivide habitualmente en raw (volcados inmutables y originales), processed (datos canónicos limpios) y extract[7]. En repositorios públicos, es imperativo utilizar exclusivamente datos públicos, sintéticos o fuertemente anonimizados para evitar brechas de cumplimiento normativo y privacidad[2].
dags/	Grafos Acíclicos Dirigidos (DAGs) de orquestación.	Directorio específico para el ecosistema de *Apache Airflow*. Define la topología, programación y dependencias de los flujos de trabajo, aislando la lógica de ejecución del momento y orden en que debe ejecutarse[5].
docs/	Documentación técnica extendida.	Contiene diagramas de arquitectura (ej. linaje de datos), diccionarios de datos, manuales de usuario y decisiones de diseño arquitectónico. Complementa al archivo principal proporcionando profundidad para revisores exhaustivos[2].
.github/	Automatización de Integración y Despliegue Continuo (CI/CD).	Define las acciones (*GitHub Actions*) para ejecutar formateo de código, validación estática (*linting*), pruebas unitarias y escaneo

		de vulnerabilidades en cada solicitud de extracción (*pull request*)[2].
.build/ o tools/	Scripts de infraestructura y aprovisionamiento.	Contiene archivos de *Docker Compose*, scripts de *PowerShell* o *Bash* necesarios para levantar y configurar el entorno de desarrollo local de manera determinista[3].
notebooks/	Exploración de datos y perfilado inicial.	Espacio reservado exclusivamente para la experimentación, análisis ad-hoc y perfilado de datos. Mantener los cuadernos aislados del código de producción (src/) preserva la limpieza y mantenibilidad del flujo de trabajo principal[2].

Archivos Fundamentales de Metadatos, Configuración y Seguridad

Más allá de la jerarquía de directorios, ciertos archivos alojados en la raíz del repositorio dictan las reglas de interacción, las dependencias del sistema y las posturas de seguridad. Estos componentes actúan como la interfaz entre el código máquina y el colaborador humano.

El archivo README.md constituye la fachada del repositorio y, frecuentemente, el único documento que un reclutador evaluará en una primera criba. Un archivo README profesional debe responder al qué, al por qué y al cómo del proyecto[3]. Su estructura debe abrir con una declaración clara del propósito del proyecto, seguida inmediatamente por un diagrama de arquitectura conciso que ilustre el linaje y flujo de los datos[2]. La literatura técnica sugiere que los detalles de configuración, los comandos de inicio rápido (quick-start), las instrucciones de pruebas y las limitaciones conocidas deben preceder al contexto de fondo, dado que los ingenieros revisores priorizan la capacidad de reproducir el entorno localmente con un solo comando[1].

La reproducibilidad del entorno se garantiza a través de archivos de manifiesto de dependencias como requirements.txt o environment.yml. Estos archivos enuncian las versiones exactas de las bibliotecas de software requeridas, mitigando el fenómeno de discrepancia de entornos ("funciona en mi máquina") y facilitando la inicialización sin fricciones por parte de terceros[2]. Además, un archivo .gitignore robusto es imperativo para evitar la inclusión accidental de archivos binarios temporales, volcados de datos masivos o, de manera crítica,

archivos de variables de entorno (.env) que contengan secretos de autenticación[2].
En el ámbito de la seguridad y el cumplimiento, la inclusión de un archivo SECURITY.md comunica las políticas de vulnerabilidad del proyecto y fomenta la divulgación responsable[3]. Complementariamente, el repositorio debe aprovechar las características de seguridad nativas de las plataformas de alojamiento, habilitando alertas de *Dependabot* para rastrear redes de dependencias vulnerables, escaneo de secretos (*Secret Scanning*) para revocar proactivamente tokens expuestos, y protección contra inserciones (*Push Protection*) para bloquear la introducción de material criptográfico confidencial en el historial de confirmaciones[2].
Finalmente, la profesionalidad de un proyecto de código abierto se consolida mediante la inclusión de archivos de gobernanza. Un archivo LICENSE aclara las restricciones y derechos legales, CONTRIBUTING.md establece los estándares para enviar mejoras y abrir incidencias, mientras que CODEOWNERS define automáticamente qué individuos o equipos son responsables de revisar modificaciones en secciones específicas del código base[3].

Estrategia de Ramificación, Confirmaciones y Descubrimiento

La mecánica de colaboración dentro del repositorio debe reflejar prácticas empresariales consolidadas. Se recomienda encarecidamente favorecer la creación de ramas (*branching*) por encima de las bifurcaciones (*forking*) para la colaboración interna, consolidando el trabajo en un repositorio único[8]. La rama principal (main o master) debe estar salvaguardada mediante reglas de protección que exijan revisiones de código y la superación de comprobaciones de estado automatizadas (pruebas de CI/CD) antes de permitir fusiones (*merges*)[8].
El historial de confirmaciones (*commit history*) debe ser semántico y limpio. Es una mala práctica utilizar mensajes de confirmación vagos; en su lugar, se debe emplear un formato consistente, utilizando el modo imperativo y limitando la longitud de la primera línea, facilitando así la legibilidad temporal del desarrollo del proyecto[9]. Asimismo, la firma de confirmaciones mediante claves GPG añade una capa de autenticidad criptográfica que protege contra manipulaciones[9]. Para maximizar la visibilidad del portafolio en los motores de búsqueda de la plataforma, el repositorio debe ser clasificado mediante etiquetas temáticas (*topics*) como data-engineering, apache-airflow, dbt o data-lakehouse, lo cual conecta el proyecto con comunidades de interés afines[9].

Fase 2: Arquitectura del Data Lakehouse Local Basado en Contenedores

La nube pública ofrece escalabilidad infinita, pero conlleva riesgos de facturación impredecible y dependencia del proveedor (*vendor lock-in*). Para demostrar la capacidad de diseñar sistemas distribuidos complejos sin incurrir en costos masivos durante la fase de portafolio, la orquestación de un *Data Lakehouse* local utilizando contenedores *Docker* representa una estrategia arquitectónica de alto calibre. Un *Data Lakehouse* fusiona la flexibilidad de almacenamiento estructurado y no estructurado de los lagos de datos con las capacidades de gestión, transaccionalidad y rendimiento analítico de los almacenes de datos tradicionales[5].
El ecosistema local se despliega típicamente a través de archivos de manifiesto

docker-compose.yaml, los cuales definen la infraestructura como código (IaC), configurando servicios interconectados, redes virtuales aisladas y volúmenes persistentes[5]. Esta topología demuestra una profunda comprensión de la separación entre los planos de almacenamiento, catálogo y cómputo.

Desacoplamiento Arquitectónico de Componentes

La infraestructura propuesta descansa sobre cuatro pilares tecnológicos de código abierto que interactúan armónicamente:

1. Capa de Almacenamiento de Objetos: MinIO En la base de la arquitectura reside MinIO, un sistema de almacenamiento de objetos distribuido de alto rendimiento, diseñado para ser estrictamente compatible con la API de Amazon S3[5]. MinIO asume el rol fundacional del lago de datos, almacenando tanto los datos crudos extraídos de sistemas transaccionales como los conjuntos de datos curados, transformados y serializados en formatos columnares altamente comprimidos, como *Apache Parquet* u ORC[5]. La adopción de MinIO en un entorno local es estratégica: permite al ingeniero de datos desarrollar y probar tuberías utilizando las bibliotecas estándar de S3 (como boto3 o conectores de AWS), garantizando que el código sea cien por ciento transferible a un entorno de nube de producción sin necesidad de refactorización[5].

2. Formato de Tabla y Gobernanza de Metadatos: Apache Iceberg y Project Nessie La simple acumulación de archivos Parquet en contenedores S3 conduce al fenómeno conocido como "pantano de datos" (*data swamp*), caracterizado por la ausencia de esquemas consistentes y la incapacidad de realizar actualizaciones concurrentes. Para remediar esto, la arquitectura introduce *Apache Iceberg*, un formato de tabla abierto que dota al almacenamiento de objetos de garantías transaccionales ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad)[5]. Iceberg gestiona los esquemas, administra particiones dinámicas y permite la evolución del esquema (añadir, renombrar o eliminar columnas) sin romper las consultas analíticas en curso ni requerir la reescritura masiva de archivos históricos[5].

Para administrar el estado global de estas tablas Iceberg, se despliega *Project Nessie* como el catálogo central del *Lakehouse*. Nessie revoluciona la gestión de metadatos al introducir semántica de control de versiones inspirada en *Git* para los datos[5]. Mediante Nessie, un ingeniero puede crear ramas (*branches*) del lago de datos completo para realizar transformaciones experimentales de forma aislada, crear puntos de control (*commits*) para auditar cambios y fijar versiones estables (*tags*) para garantizar la reproducibilidad de los modelos analíticos y de *Machine Learning*[5].

3. Motor de Cómputo Distribuido y Análisis: Trino (o Apache Spark) Con los datos almacenados en MinIO y estructurados por Iceberg, se requiere un motor para procesar y consultar la información. *Trino* es un motor de consultas SQL masivamente paralelo (MPP) y distribuido, optimizado para federar consultas sobre petabytes de datos alojados en almacenamientos heterogéneos[5]. Trino se conecta al catálogo de Nessie para descubrir las tablas Iceberg y ejecuta consultas SQL estándar directamente contra los archivos subyacentes en MinIO, eliminando la necesidad de mover o copiar datos a bases de datos propietarias[5]. Su arquitectura nativa de la nube permite configurar un nodo coordinador y múltiples nodos

trabajadores (*workers*) dentro del entorno Docker para demostrar escalabilidad horizontal[5]. Alternativamente, o en paralelo, se puede orquestar un clúster de *Apache Spark* para gestionar cargas de trabajo de extracción, carga y transformación (ELT) que requieran manipulación de datos pesada en memoria[1].

4. Capa de Modelado Dimensional: Arquitectura Medallón El flujo de los datos a través de esta infraestructura se adhiere a la Arquitectura Medallón, un patrón de diseño industrial que estructura el lago de datos en zonas de refinamiento progresivo[5]:

- **Zona Bronce (Landing/Ingestión):** Recepción de datos crudos e inmutables, volcados directamente a *buckets* de MinIO (por ejemplo, archivos CSV, JSON de APIs o flujos de Kafka) conservando su fidelidad original[1].
- **Zona Plata (Staging/Limpieza):** Datos filtrados, estandarizados, con tipos de datos corregidos y almacenados en formato columnar estructurado mediante Iceberg. En esta fase se resuelven anomalías sintácticas[1].
- **Zona Oro (Curated/Consumo):** Modelos de negocio altamente refinados, organizados en esquemas de estrella (tablas de hechos y dimensiones) mediante transformaciones SQL pesadas, listos para alimentar cuadros de mando de inteligencia de negocios o modelos predictivos[1].

Fase 3: Orquestación de Flujos de Trabajo con Apache Airflow

Una arquitectura de datos robusta carece de valor operacional sin un sistema de orquestación que gestione el ciclo de vida de los datos, resuelva las dependencias topológicas y maneje la recuperación ante errores. *Apache Airflow*, desarrollado originalmente por Airbnb y cedido a la Fundación Apache, es el estándar de facto en la industria para programar, crear y monitorear flujos de trabajo de manera programática utilizando Python[13].

En el contexto del portafolio, Airflow asume el rol de director de orquesta. Sus flujos de trabajo se definen como Grafos Acíclicos Dirigidos (DAGs), donde cada nodo representa una unidad discreta de trabajo (una tarea) y los bordes representan las dependencias cronológicas o lógicas entre ellas[1]. La integración de Airflow en la pila de Docker requiere un contenedor para el programador (*scheduler*), otro para la interfaz de usuario web (*webserver*), y una base de datos relacional transaccional (comúnmente PostgreSQL) que almacena los metadatos de las tareas, los historiales de ejecución y la gestión de usuarios[11].

Patrones de Diseño en la Orquestación

La construcción de DAGs para el portafolio debe demostrar dominio sobre conceptos avanzados de orquestación:

1. **Integración Dinámica e Idempotencia:** Las canalizaciones deben diseñarse bajo el principio de idempotencia; es decir, ejecutar el mismo DAG múltiples veces para un intervalo de tiempo específico (por ejemplo, reprocesar los datos de ayer debido a un fallo) debe producir exactamente el mismo resultado final, sin duplicar registros ni corromper el estado del lago de datos[1]. Esto se logra mediante el uso juicioso de particiones de fecha (ej. ingest_date=YYYY-MM-DD) en las rutas de MinIO y el uso de

operaciones de sobrescritura condicional[11].

2. **Sensores y Proveedores Externos:** Para que Airflow se comunique fluidamente con MinIO (S3) y Trino, el entorno debe estar provisto de paquetes específicos como apache-airflow-providers-amazon[13]. Esto permite el uso de operadores nativos y sensores que, por ejemplo, pausan un flujo de trabajo hasta que un archivo específico aterriza en un *bucket* de MinIO antes de desencadenar la fase de transformación[12].
3. **Transformaciones As-a-Code con dbt:** En lugar de codificar complejas transformaciones de datos dentro de los operadores de Python de Airflow, la mejor práctica contemporánea delega el modelado SQL a *dbt* (data build tool). Airflow actúa simplemente como el disparador, utilizando operadores personalizados o contenedores efímeros para invocar compilaciones de dbt que ejecutan modelos modulares y versionados directamente en el motor de Trino, promoviendo los datos desde la capa Bronce hasta la capa Oro[1].
4. **Gestión Centralizada de Logs:** Una demostración de diseño de sistemas distribuidos avanzados consiste en configurar Airflow para enrutar sus propios registros de ejecución de tareas (*task logs*) hacia un *bucket* dedicado en MinIO[13]. Esto desacopla el almacenamiento de los registros de los contenedores de cómputo efímeros, garantizando que el historial de auditoría persista incluso si la infraestructura subyacente es destruida y reconstruida[13].

Fase 4: Calidad de Datos (Data Quality) y Pruebas Rigurosas

En el paradigma moderno de la ingeniería de datos, el impacto de una tubería defectuosa o de la propagación de datos corruptos es asimétricamente negativo en comparación con la ausencia de datos. Históricamente, los ingenieros reaccionaban a los errores reportados por los analistas. Hoy, el estándar exige la validación proactiva y automatizada de cada faceta del sistema[6].

Este proyecto de portafolio abordará la calidad y confiabilidad del software mediante un enfoque bifronte: pruebas del código de infraestructura empleando pytest, y validaciones del estado y semántica de los datos en movimiento utilizando el marco Great Expectations.

Pruebas de Software Determinista con Pytest

El marco pytest se utiliza para formular pruebas sobre la lógica del código de las tuberías. Según la heurística del dominio, si una verificación de error fallaría de manera idéntica utilizando datos de dos días diferentes, es un problema de código y pertenece al ámbito de Pytest[6]. El objetivo es demostrar que las transformaciones manejan correctamente las tipologías de datos, los valores nulos, las reglas de negocio, los números negativos y las codificaciones inusuales[6].

- **Pruebas Unitarias Aisladas:** Evalúan funciones de transformación de Python en un entorno de laboratorio. Se construyen *DataFrames* en memoria pequeños y controlados manualmente, y se verifica que una función de limpieza específica emita la salida esperada sin invocar bases de datos, redes o sistemas de archivos externos[6]. Esto

mantiene las pruebas extremadamente rápidas y exentas de dependencias[6].

- **Pruebas de Integración de Tuberías:** Las pruebas de integración aseguran que la comunicación entre módulos funciona adecuadamente. Dado que configurar contextos de bases de datos para cada prueba es computacionalmente costoso, se emplea el concepto de *Fixtures* de Pytest[6]. Una *fixture* con alcance de sesión (@pytest.fixture(scope="session")) permite, por ejemplo, inicializar un clúster de PySpark de un solo nodo al principio de la ejecución de las pruebas y compartir esa sesión a través de múltiples casos de prueba, optimizando el tiempo de ejecución[6].
- **Manejo de Casos Límite:** El decorador @pytest.mark.parametrize es una herramienta invaluable para probar un modelo contra una matriz de entradas y resultados esperados[6]. Permite validar el comportamiento del código frente a múltiples perturbaciones simuladas sin necesidad de escribir funciones de prueba duplicadas y verbosas, garantizando que el código sea resistente a las anomalías de los datos reales[6].

Validación del Estado de los Datos con Great Expectations (GX)

Mientras que Pytest comprueba que el código funcionará asumiendo que los datos de entrada son correctos, *Great Expectations* (GX) asume la responsabilidad complementaria: audita los datos que fluyen por la tubería para garantizar que se ajustan a contratos predefinidos antes de causar estragos en los modelos analíticos[6]. GX es un marco de validación declarativo y de código abierto que se integra directamente en los procesos ETL/ELT[16].
La arquitectura de GX se basa en una jerarquía de configuración estricta que el ingeniero debe dominar:

Concepto de GX	**Definición y Mecanismo Funcional**	**Implementación Práctica**
Data Context	Es el cerebro organizativo del marco. Mantiene un registro centralizado de la configuración, conexiones y ubicaciones de metadatos, actuando como el punto de entrada unificado para todas las operaciones de validación[16].	Se inicializa vía CLI (great_expectations init) y genera una estructura de carpetas que debe someterse a control de versiones en Git[18].
Data Source & Asset	Define la conexión al sistema subyacente. Puede ser un archivo CSV local, un DataFrame de Pandas en memoria, o una conexión SQL a sistemas como	Se configura programáticamente utilizando métodos como add_pandas_gcs o equivalentes de conectividad de bases de

	Postgres, Snowflake, Trino o archivos S3/MinIO[17].	datos[17].
Batch Definition	Representa un subconjunto lógico y discreto de un *Data Asset* que se someterá a validación en una ejecución específica (por ejemplo, la partición de datos ingeridos "hoy")[17].	Permite parametrizar las validaciones para que la tubería pueda verificar volúmenes enormes procesando solo los deltas diarios[17].
Expectations	Reglas declarativas que dictan cómo debe lucir "datos buenos". Ejemplos incluyen expect_column_values_to_not_be_null o expect_column_values_to_be_between[16].	GX ofrece más de 100 expectativas integradas que verifican esquemas, rangos numéricos, patrones regex, unicidad y detección de valores faltantes[16].
Expectation Suites	Una colección coherente de expectativas agrupadas lógicamente[16]. Actúan como contratos de calidad de datos, versionados en formato JSON o YAML[16].	Definidas por los expertos del negocio y codificadas por el ingeniero. Ej: "Flight_Expectation_Suite" que agrupa validaciones para datos aeronáuticos[17].
Checkpoints	El motor de ejecución que toma una *Expectation Suite*, la lanza contra un *Batch* de datos específico y calcula los resultados[16].	Se integra nativamente dentro de un operador de Airflow. Si el *Checkpoint* falla, la ejecución del DAG se detiene[16].
Data Docs	Documentación HTML estática y legible por humanos generada automáticamente que visualiza el éxito o fracaso de las validaciones a lo largo del tiempo[16].	Puente de comunicación vital que transforma comprobaciones de ingeniería oscuras en paneles de control comprensibles para analistas de negocios[16].

El flujo de trabajo operativo exige que el proyecto incluya un paso donde, justo después de que Airflow cargue los datos crudos en la zona Bronce y antes de modelarlos en la zona Plata, se

invoque un *Checkpoint* de GX. Si una tabla crítica presenta nulos en sus claves primarias, la tubería debe fallar explícitamente, emitiendo alertas y previniendo la contaminación de la capa analítica[16]. Esto transforma la ingeniería de datos de un paradigma de "lucha reactiva contra incendios" a uno de "aseguramiento proactivo de la calidad"[16].

Fase 5: Análisis de Latencia de Red para Modelado de Migración a la Nube

Un aspecto que distingue profundamente a un ingeniero *Senior* de uno *Junior* es la consideración del impacto geoespacial y físico de la infraestructura sobre el rendimiento del sistema. Aunque el portafolio se desarrolle en un *Data Lakehouse* local para mitigar costos, debe incluir un estudio arquitectónico formal sobre cómo se desplegaría este sistema en la nube pública (AWS, Azure o GCP), justificando la selección de la región de alojamiento.
La latencia de red, habitualmente medida como el Tiempo de Ida y Vuelta (*Round Trip Time* o RTT), representa el retraso en milisegundos (ms) que tarda un paquete de datos en viajar desde el cliente (usuario, aplicación o dispositivo de origen) hasta el servidor en la nube y retornar[21]. Este retraso está dictado por las inmutables leyes de la física (la velocidad de la luz en la fibra óptica), pero se ve severamente agravado por el enrutamiento de los proveedores de servicios de Internet (ISPs), las relaciones de emparejamiento de red (*peering*), el número de saltos de enrutadores intermedios y la congestión general de las líneas[22].

Impacto de la Latencia en las Cargas de Trabajo Analíticas

La selección de la región impacta directamente la usabilidad del sistema. Para justificar una elección de infraestructura, el proyecto debe categorizar las cargas de trabajo objetivo y aplicar umbrales de tolerancia de latencia:

Rango de Latencia (RTT)	**Viabilidad y Tipología de Carga de Trabajo Asociada**	**Consecuencias Arquitectónicas**
Menos de 50 ms	Excelente. Requisito estricto para aplicaciones interactivas en tiempo real, inferencia de *Machine Learning* sincrónica, juegos multijugador y consultas transaccionales de alta frecuencia[21].	Difícil de lograr a distancias intercontinentales. Requiere el despliegue de infraestructura en regiones geográficamente limítrofes o el uso exhaustivo de redes de entrega de contenido (CDN) y nodos perimetrales (*Edge computing*).
50 ms - 100 ms	Aceptable. Rango estándar	Los usuarios perciben un

	para aplicaciones web corporativas, cuadros de mando interactivos de Inteligencia de Negocios y operaciones de bases de datos relacionales convencionales[21].	retraso mínimo pero tolerable. Es el objetivo general para plataformas SaaS y despliegues de almacenes de datos analíticos regionales.
Más de 100 ms	Degradado para uso interactivo. Produce una fricción perceptible que afecta negativamente la experiencia del usuario final[21].	Restringido a cargas de trabajo no interactivas, transferencias masivas de archivos asincrónicas, procesamientos por lotes (*batch processing*), copias de seguridad de recuperación ante desastres (DR) y repositorios de archivo a largo plazo[21].

Metodología de Evaluación Multicloud (Caso Práctico: Costa Rica)

El portafolio debe incorporar un documento de investigación empírica (incluido en el directorio /docs/) que aplique herramientas de *Looking Glass* y pruebas de alcance. Si asumimos un centro de operaciones basado en Costa Rica, el ingeniero debe ejecutar sondeos de red utilizando herramientas públicas consolidadas:

- **Amazon Web Services (AWS):** Utilizando utilidades como *Cloudping* o el *Amazon EC2 Reachability Test*, se miden los RTT hacia las regiones de Estados Unidos. Históricamente, las regiones us-east-1 (Norte de Virginia), us-east-2 (Ohio) y us-west-1 (Norte de California) ofrecen los mejores resultados para Centroamérica, frecuentemente oscilando en la banda de 60-90 ms dependiendo del enrutamiento de tránsito submarino versus emparejamiento directo[22].
- **Microsoft Azure:** A través de *Azure Speed Test*, que emplea solicitudes HTTPS desde el navegador a puntos finales de almacenamiento, o utilizando la herramienta TCP psping para aislar la latencia de red pura sin la sobrecarga del protocolo de aplicación HTTP. Esto permite identificar la latencia hacia las regiones del Este y Sur de EE.UU[21]. Las variaciones mayores al 30% en diferentes momentos del día pueden indicar congestión de tránsito por parte de terceros[22].
- **Google Cloud Platform (GCP):** Google ofrece herramientas como *GCPing* para medir la latencia mediana global[25]. Además, dispone del *Region Picker*, una utilidad avanzada que asiste a los arquitectos equilibrando tres vectores fundamentales: la latencia de red, los costos de infraestructura computacional por hora de vCPU, y la intensidad de carbono (gCO2eq/kWh) de la red eléctrica que alimenta el centro de datos[26].

La síntesis de esta investigación en el informe del portafolio demostrará concluyentemente que el candidato no implementa soluciones en regiones arbitrarias, sino que optimiza para ofrecer el mejor equilibrio de precio, sostenibilidad y experiencia de usuario.

Fase 6: Gobernanza de la Inteligencia de Negocios con Power BI (PBIP) y TMDL

El ciclo de vida de los datos culmina en la visualización, la capa donde la lógica de negocio se transforma en inteligencia accionable para los tomadores de decisiones. Históricamente, este paso ha representado un punto de fricción masivo en los paradigmas *DataOps* y CI/CD. Las herramientas analíticas convencionales guardaban sus desarrollos en archivos binarios monolíticos y opacos (como el formato .pbix heredado de Power BI)[28]. Estos formatos binarios impedían a los equipos rastrear cambios granulares mediante Git, dificultaban las fusiones (*merges*) simultáneas de múltiples analistas, y oscurecían el linaje de los modelos semánticos[29]. Para resolver este cuello de botella y unificar el desarrollo analítico con las prácticas rigurosas de la ingeniería de software, el proyecto de portafolio debe adoptar el estándar de **Proyecto de Power BI** (.pbip)[28].

Desacoplamiento Analítico y Control de Versiones

Al guardar un cuadro de mando como un proyecto .pbip, el entorno extrae las configuraciones incrustadas dentro del binario opaco y las serializa, separando el ecosistema en archivos de texto legibles, dividiendo lógicamente el modelo semántico de la representación visual del informe[28].

Esta arquitectura modular está impulsada por dos componentes técnicos primordiales:

1. **Tabular Model Definition Language (TMDL):** Es el avance más significativo en la gestión semántica. TMDL reemplaza al antiguo y verboso archivo JSON model.bim[30]. Serializa el modelo de datos en múltiples archivos de texto plano, agrupando aproximadamente un archivo por cada tabla lógica del esquema[30]. Esta estructura jerárquica significa que las medidas formuladas en código DAX (*Data Analysis Expressions*), las relaciones inter-tablas, las particiones y las reglas de seguridad a nivel de fila (RLS) pueden ser versionadas semánticamente[36]. Un ingeniero de datos puede abrir una solicitud de extracción en GitHub modificando una sola línea de una métrica DAX, y los revisores podrán examinar exactamente ese cambio en texto puro, aprobándolo e integrándolo a la rama principal de producción sin temor a corromper el modelo entero[30].
2. **Formato de Reporte de Power BI (PBIR):** De forma paralela al modelo de datos, la capa de visualización se almacena utilizando el estándar PBIR[29]. Este formato documenta en texto la configuración precisa de los lienzos, temas visuales, filtros interactivos y disposición geométrica de los gráficos en el cuadro de mando[32].

Incluir un directorio (por ejemplo, src/visualization/dashboard.pbip) gestionado íntegramente a través de TMDL y PBIR posiciona el portafolio a la vanguardia de la industria, demostrando cómo extender la gobernanza continua (CI/CD) desde las canalizaciones de ingesta iniciales hasta la visualización final del negocio[28].

Fase 7: Plan de Acción Detallado y Construcción de Archivos

Para materializar este diseño arquitectónico teórico en un activo evaluable que asombre a los evaluadores técnicos, se debe seguir una ruta crítica de desarrollo. La secuencia de construcción asegura que cada módulo tenga las dependencias resueltas antes de su configuración.

A continuación, se define el inventario secuencial de archivos y configuraciones que deben crearse:

Bloque 1: Repositorio e Inicialización Base

El fundamento del proyecto requiere establecer el perímetro de seguridad y gobernanza.

Ruta y Archivo a Crear	**Contenido y Acción a Desarrollar**
README.md	Redactar el documento central. Incluir diagramas de arquitectura (generados con herramientas como *Draw.io* o *Mermaid*), apilar las insignias (*badges*) de estado de CI/CD, enumerar las tecnologías utilizadas, y escribir una sección clara de "Cómo ejecutar localmente" en menos de 5 comandos[2].
.gitignore	Configurar reglas estrictas para rechazar entornos virtuales de Python (venv/), almacenes temporales locales, volcados masivos de datos (*.csv, *.parquet), claves secretas (.env) y metadatos del sistema operativo (.DS_Store)[2].
requirements.txt	Fijar versiones inmutables (ej. pandas==2.1.0, apache-airflow==2.7.0, great-expectations, pytest) para garantizar que la replicación del entorno funcione sin errores de resolución de dependencias[4].
.github/workflows/ci-cd.yaml	Codificar la integración continua. Escribir trabajos (*jobs*) que inicialicen una instancia de Ubuntu, instalen Python, ejecuten linters (como flake8 o black) e invoquen la suite de pruebas unitarias de pytest ante cada

	empuje a la rama principal[2].
SECURITY.md	Detallar instrucciones breves para reportar brechas de seguridad (como credenciales expuestas) a través del canal privado del repositorio en lugar de abrir una incidencia pública[3].

Bloque 2: Infraestructura y Contenedores (*Lakehouse*)

La segunda fase orquesta el entorno distribuido local simulando la infraestructura de nube.

Ruta y Archivo a Crear	**Contenido y Acción a Desarrollar**
docker-compose.yaml	Archivo manifiesto principal. Definir servicios para MinIO (almacenamiento), servidor PostgreSQL (metadatos para Nessie y Airflow), Project Nessie (catálogo), el nodo coordinador de Trino, y nodos trabajadores escalables[5]. Configurar las redes puente aisladas (external: true)[11].
docker-compose.airflow.yaml	Archivo superpuesto (*overlay file*). Configurar los servicios de Airflow (Webserver, Scheduler, y Base de Datos local) y enrutarlos para compartir la misma red virtual de los componentes del *Lakehouse*, asegurando comunicación transparente mediante DNS de contenedores[11].
Dockerfile (Airflow)	Extender la imagen oficial de Airflow. Incorporar utilidades del sistema operativo e instalar bibliotecas críticas, específicamente proveedores adicionales como apache-airflow-providers-amazon, conectores de Trino y dependencias de dbt/Great Expectations[5].
.env.example	Plantilla de credenciales. Proveer variables base no funcionales como MINIO_ROOT_USER=admin,

	AIRFLOW_CONN_MINIO=... para guiar al evaluador sin exponer contraseñas reales[2].

Bloque 3: Orquestación, Ingesta y Modelado

Con la infraestructura lista, se deben desarrollar las canalizaciones programáticas.

Ruta y Archivo a Crear	Contenido y Acción a Desarrollar
src/ingestion_module.py	Escribir funciones puras de extracción de datos, limpieza sintáctica temprana y uso de clientes compatibles con S3 (boto3 o minio_client) para escribir los datos inmutables hacia la capa Bronce[2].
dags/etl_pipeline_dag.py	Definir el grafo principal en Python. Programar los intervalos de ejecución (schedule_interval), establecer las dependencias posicionales (task_1 >> task_2), y configurar el manejo de errores (políticas de reintentos escalonados)[5].
sql/bronze_to_silver.sql	Desarrollar consultas estructuradas en sintaxis Trino o construir modelos en dbt. Estas consultas leerán los archivos brutos y crearán tablas estructuradas en formato Iceberg, aplicando *casts* de tipos de datos, limpieza de valores anómalos y deduplicación temporal[1].
sql/silver_to_gold.sql	Construir agregaciones complejas, uniones (*joins*) de tablas de dimensiones y hechos, y lógicas de agregación ventanadas, dejando los datos en el esquema curado listo para su ingestión por los cuadros de mando[1].

Bloque 4: Pruebas de Software y Aseguramiento de Datos

La institucionalización de la calidad y resiliencia en el proceso de datos.

Ruta y Archivo a Crear	Contenido y Acción a Desarrollar

tests/conftest.py	Definir *fixtures* de Pytest. Instanciar *DataFrames* de prueba pre-cargados y configurar clientes de base de datos simulados (o *mocks* de red) compartidos globalmente durante la sesión de pruebas[2].
tests/test_transforms.py	Redactar múltiples pruebas con afirmaciones (assert). Emplear @pytest.mark.parametrize para someter las funciones lógicas de src/ a múltiples combinaciones de casos límite (tipos de datos inesperados, números negativos) para asegurar que el código es determinista y no se rompe bajo estrés[2].
Directorio great_expectations/	Desplegar el marco a través de CLI. Modificar great_expectations.yml para vincular los motores de ejecución (conectores Trino o MinIO Pandas). Configurar una *Expectation Suite* en JSON detallando reglas de negocio imperativas (no-nulos, intervalos, diccionarios predefinidos)[16].
dags/data_quality_dag.py	Integrar el operador de *Checkpoint* de Great Expectations dentro del flujo de Airflow. Configurar alertas para que, en caso de fallar el contrato semántico de los datos, la tubería detenga su avance e impida la contaminación downstream[16].

Bloque 5: Visualización de Negocio (PBIP)

La generación de inteligencia e integración de los analistas al ciclo de vida de desarrollo.

Ruta y Archivo a Crear	**Contenido y Acción a Desarrollar**
src/visualization/dashboard.pbip	Cuadro de mando generado localmente conectado vía ODBC/DirectQuery a la capa Oro administrada por Trino. Al guardarlo explícitamente en formato de Proyecto, se generarán paralelamente las estructuras de

	texto de metadatos[28].
*.tmdl / *.pbir (autogenerados)	Subir estas carpetas de texto al repositorio de GitHub. El evaluador podrá leer el código fuente de los modelos tabulares, fórmulas de agrupamiento y estructuras DAX, verificando la competencia del candidato en modelado dimensional sin necesidad de instalar o abrir software propietario[29].

Fase 8: Áreas de Investigación y Documentación Estratégica

Para que el proyecto trascienda un simple ejercicio de codificación y asuma la profundidad de un sistema de grado empresarial, el candidato debe realizar e integrar las siguientes investigaciones analíticas en los manuales técnicos del directorio /docs/:

1. **Economía Computacional del Lakehouse frente al Data Warehouse:** Elaborar un análisis técnico sobre cómo la disociación del cómputo (Trino/Spark) del almacenamiento de objetos (MinIO/S3) reduce dramáticamente la facturación en la nube, en contraposición al modelo monolítico clásico de almacenamiento y cómputo combinados. Argumentar cómo el empleo de formatos de tabla abiertos (Iceberg) evita el bloqueo de formato patentado de los proveedores, dotando al sistema de portabilidad y mitigando el *vendor lock-in*[5].
2. **Topología de Redes Geográficas y Optimización de Latencia:** Como se delineó en la Fase 5, documentar un mapeo empírico de latencia (usando *GCPing* o *Cloudping*) desde la ubicación local (ej. Centroamérica) hacia infraestructuras en Virginia, Ohio, o zonas adyacentes. Presentar la investigación justificando de manera cuantitativa qué región y qué proveedor de nube minimizaría el tiempo de retorno para los analistas interactivos locales versus los procesos extractivos nocturnos[21].
3. **Tolerancia a Fallos, Mecanismos de Reintento y Estado:** Investigar e ilustrar los patrones de diseño subyacentes en la ejecución de Airflow. Detallar por escrito el concepto de *Idempotencia* y explicar cómo la configuración cuidadosa de las políticas de retención, las particiones de los datos procesados, y las reanudaciones desde los puntos de fallo previenen la duplicación y corrupción silenciosa de datos[1].
4. **Mutabilidad sobre Datos Inmutables (Iceberg):** Presentar pruebas teóricas y scripts que demuestren cómo Apache Iceberg abstrae la complejidad de la actualización de datos (operaciones UPSERT o DELETE). Explicar cómo el sistema mantiene el rendimiento evitando el *Data Swamp* mediante compactaciones periódicas y control de instantáneas temporales (*Time Travel*), lo cual es imposible con sistemas S3 sin gestionar[5].

Conclusión

El diseño riguroso y la ejecución meticulosa de un portafolio de ingeniería de datos requieren conjugar múltiples disciplinas técnicas con una visión arquitectónica clara. Este marco de acción exhaustivo supera los límites de los proyectos tutoriales aislados, entrelazando las mejores prácticas contemporáneas desde la ingestión de datos crudos hasta la cristalización analítica visual.

Al establecer primero un entorno gobernado, estructurado y automatizado en GitHub, se comunica profesionalismo e higiene de código. La implantación de una infraestructura agnóstica a la nube, orquestada localmente mediante contenedores Docker, MinIO, Trino y formatos universales como Iceberg, evidencia capacidades de diseño distribuido resiliente. La integración simbiótica de validaciones deterministas algorítmicas de software (*Pytest*) junto con contratos inflexibles sobre el estado empírico de la información (*Great Expectations*) demuestra una cultura de madurez orientada hacia el aseguramiento continuo de la calidad de datos en los flujos gobernados por Airflow.

Finalmente, la integración del modelado analítico a través de las extensiones textuales de *Power BI (PBIP)* rompe con los silos tradicionales, llevando a toda la empresa bajo el amparo de la revisión colaborativa. Adicionado a los profundos análisis de latencia en la nube multirregional que justifican el despliegue geográfico de estas cargas de trabajo, el candidato generará un artefacto técnico que no solo funciona irreprochablemente a escala, sino que solidifica de manera inapelable su posición como ingeniero analítico frente a los estándares más exigentes de la industria global.

Obras citadas

1. 20 GitHub Repos to Learn Data Engineering (With Real Projects), https://medium.com/@reliabledataengineering/20-github-repos-to-learn-data-engineering-with-real-projects-f90ea6ed375e
2. GitHub for Data Engineers: Present Projects That Get Read, https://dataengineeracademy.com/blog/github-for-data-engineers-how-to-present-your-projects/
3. GitHub Repository Structure Best Practices | by Soulaiman Ghanem, https://medium.com/code-factory-berlin/github-repository-structure-best-practices-248e6effc405
4. Best practices for structuring AI or ML projects on GitHub #189034, https://github.com/orgs/community/discussions/189034
5. GitHub - vutrinh274/local_lakehouse, https://github.com/vutrinh274/local_lakehouse
6. Testing Data Pipelines Like You Mean It: A pytest Crash Course for, https://dev.to/nbaubek/testing-data-pipelines-like-you-mean-it-a-pytest-crash-course-for-data-engineers-24
7. A project structure for doing and sharing data engineer work. · GitHub, https://github.com/Chek0rrdn/DataEngineer_ETL
8. Best practices for repositories - GitHub Docs, https://docs.github.com/en/repositories/creating-and-managing-repositories/best-practices-for-repositories

9. GitHub Repository Best Practices - DEV Community, https://dev.to/pwd9000/github-repository-best-practices-23ck
10. Unlocking Data Locally: An End-to-End On-Premises Data Stack, https://www.bi-solutions-hamburg.com/blog/self-hosted-data-lakehouse
11. Open Source Data Lake 7: Automating Data Ingestion with Airflow, https://medium.com/@marekczarnecki_50908/oss-data-lake-7-automating-data-ingestion-with-airflow-a6648dcf303a
12. How to build a data lake from scratch - Part 2: Connecting the, https://towardsdatascience.com/how-to-build-a-data-lake-from-scratch-part-2-connecting-the-components-1bc659cb3f4f/
13. Using Apache Airflow with MinIO, https://www.min.io/blog/apache-airflow-minio
14. Build your own Data Lake on your infrastructure | by Haq Nawaz, https://aws.plainenglish.io/build-your-own-data-lake-on-your-infrastructure-c5015658b237
15. Building a Modern Data Lake with Minio, Spark, Airflow via Docker., https://github.com/le-oasis/docker-airflow-spark
16. Mastering Data Quality with Great Expectations: Complete Guide for, https://www.datatoinsights.ai/blog/great-expectations-the-complete-guide-to-ensuring-data-quality-in-modern-data-pipelines-baac3d
17. Mastering Data Quality in ETL Pipelines with Great Expectations, https://anujsyal.com/mastering-data-quality-in-etl-pipelines-with-great-expectations
18. Ensuring Data Quality, With Great Expectations, https://www.startdataengineering.com/post/ensuring-data-quality-with-great-expectations/
19. Great Expectations Tutorial: Validating Data with Python - DataCamp, https://www.datacamp.com/tutorial/great-expectations-tutorial
20. Great Expectations | OpenMetadata Data Quality Integration, https://docs.open-metadata.org/v2.0.x/connectors/ingestion/great-expectations
21. Azure Latency Test | Measure Datacenter Latency, https://www.azurespeed.com/Azure/Latency
22. Measuring network latency to AWS Region before deployment, https://aws.amazon.com/blogs/networking-and-content-delivery/measuring-network-latency-to-aws-region-before-deployment/
23. AWS Latency Monitoring, https://www.cloudping.co/
24. cloudping.info, https://www.cloudping.info/
25. GCPing.com, https://www.gcping.com/
26. Global Locations - Regions & Zones - Google Cloud, https://cloud.google.com/about/locations
27. Google Cloud region picker, https://cloud.withgoogle.com/region-picker/
28. Power BI Project (PBIP) | Data Dictionary - Data Panda, https://datapanda.eu/en/dictionary/power-bi-project-pbip
29. PBIP GA and PBIR Default: What Changes in Power BI - Solv Systems, https://solv-systems.com/resources/power-bi-pbip-pbir
30. What Is a PBIP File in Power BI, and What Is TMDL? - Express Lint,

https://www.expresslint.com/powerbi-guides/what-is-pbip-and-tmdl/
31. Power BI File Formats Explained: PBIX vs PBIT vs PBIP vs PBIR, https://radacad.com/power-bi-file-formats-explained-pbix-vs-pbit-vs-pbip-vs-pbir/
32. Power BI PBIP & PBIR Explained - KS AI & Cloud Solutions, https://ksacsolutions.com/blog/power-bi-pbip-pbir-explained/
33. Power BI Desktop projects (PBIP) - Microsoft Learn, https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview
34. What is PBIP file format in Power BI? - YouTube, https://www.youtube.com/watch?v=CsBifFPMQ4s
35. Power BI Desktop project semantic model folder - Microsoft Learn, https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset
36. Power BI Project (PBIP) is now generally available - here's how it, https://www.red-gate.com/simple-talk/data-analytics/powerbi/power-bi-project-pbip-format-guide/